

Terraform-Challenge-3

1. Goal of the Exercise

The goal of this exercise is to test the understanding of a Cloud Compute deployment with Terraform.

The task is to create a codebase for a microservice deployment on a cloud environment using Terraform. Since the focus is not the application itself, the service is a simple web server that returns a static page.

1.1 Requirements — Service Details

- The service will be a simple web server that returns a static page.
- apache2 or nginx may be used to serve a simple HTML file.
- The service should be containerized and stored in a Docker registry.
- The service should be deployed on a single VM instance as a Docker container.
- The image should be pulled from the registry specified earlier.
- The service should not be accessible from the internet but only through a Load Balancer, except for SSH access from specific IP addresses.

1.2 Deliverables

1. Dockerfile to build the image.
2. Terraform codebase to deploy the Docker registry, the VPC and all required networking resources, the EC2 instance, and the Load Balancer.

1.3 Notes from the Brief

- Since a CI/CD pipeline is not part of this exercise, this is worked around with the help of Terraform.
- To push the image to the Docker registry, the image is built locally before running the Terraform code.
- Once the image registry is created, Terraform is used to push the local image to the registry.
- Similarly, Terraform is used to pull the image from the registry and run it on the VM instance once it is created.
- For SSH access: use a locally created key, add it to the cloud provider key storage, and attach it to the instance.
- Use Terraform best practices for file structure and code organization — variables, locals, modules, etc. where appropriate.
- Include all tfvars, scripts and other helper files used to deploy the application.
- Any cloud provider may be used, although AWS is preferred.
- Use a free tier or trial account to deploy the resources.

2. Solution Overview

The solution deploys a containerised nginx microservice on AWS. The container image is built locally from a Dockerfile, then Terraform creates an Amazon ECR repository, tags the local image for that repository and pushes it. Terraform then builds the network, launches a single EC2 instance in a private subnet, and the instance's user-data script pulls the image back down from ECR and runs it as a Docker container on port 80.

The instance has no public IP address. The only inbound paths to it are HTTP traffic from the Application Load Balancer's security group and SSH from one whitelisted IP address, which satisfies the "not accessible from the internet except through the Load Balancer" requirement.

2.1 Request Flow

```
Internet user
  |
  v
Application Load Balancer (internet-facing, HTTP :80)
- lives in public subnet 1 (us-east-1a) and public subnet 2 (us-east-1b)
- security group terraform-alb-sg : inbound 80 from 0.0.0.0/0
  |
  | forwards to target group terraform-microservice-tg (HTTP :80)
  v
EC2 instance terraform-microservice-ec2 (t3.micro, private subnet 10.0.10.0/24)
- NO public IP
- security group terraform-ec2-sg :
  inbound 80 <- source = terraform-alb-sg (ALB only)
  inbound 22 <- source = 49.43.224.127/32 (admin IP only)
  |
  v
Docker container "microservice" -> nginx:alpine -> index.html

Outbound path for the image pull:
EC2 (private subnet) -> NAT Gateway (public subnet 1) -> Internet Gateway -> Amazon ECR
```

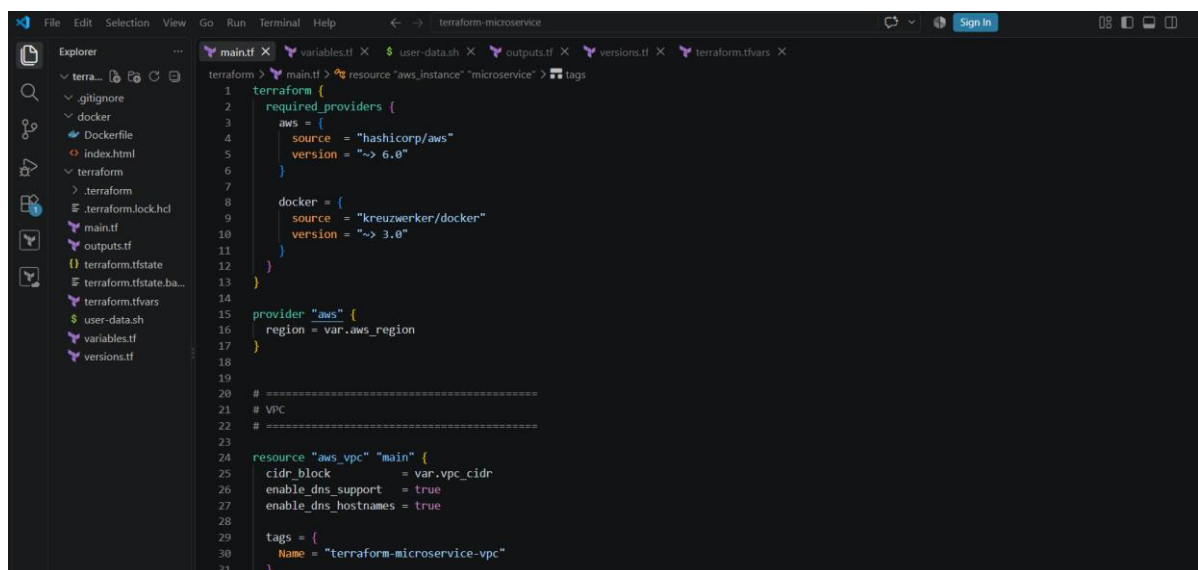
2.2 Resources Created by Terraform

Layer	Resources
Registry	aws_ecr_repository.microservice — terraform-microservice, scan on push enabled
Image push	docker_image, docker_tag, docker_registry_image (kreuzwerker/docker provider)
Networking	VPC 10.0.0.0/16, 2 public subnets, 1 private subnet, Internet Gateway, NAT Gateway + EIP, public and private route tables and associations
Security	terraform-alb-sg, terraform-ec2-sg, IAM role + instance profile, SSH key pair
Compute	aws_instance.microservice — t3.micro, private subnet, user-data bootstrap
Load balancing	aws_lb (ALB), aws_lb_target_group, aws_lb_target_group_attachment, aws_lb_listener

2.3 Repository / File Structure

Terraform best practices are followed by splitting configuration by concern — providers and resources in `main.tf`, all inputs in `variables.tf`, all environment values in `terraform.tfvars`, all outputs in `outputs.tf`, and the instance bootstrap in a separate templated shell script.

```
terraform-microservice/
├── docker/
│   ├── Dockerfile          # builds the nginx image
│   └── index.html           # the static page served by the container
├── terraform/
│   ├── main.tf              # providers + all AWS and Docker resources
│   ├── variables.tf         # input variable declarations
│   ├── terraform.tfvars    # environment-specific values
│   ├── outputs.tf          # ALB DNS, ECR URL, VPC ID, EC2 private IP
│   ├── versions.tf         # provider / Terraform version constraints
│   └── user-data.sh         # EC2 bootstrap: install Docker, pull from ECR, run
└── .gitignore              # excludes .terraform/, *.tfstate, *.pem
```



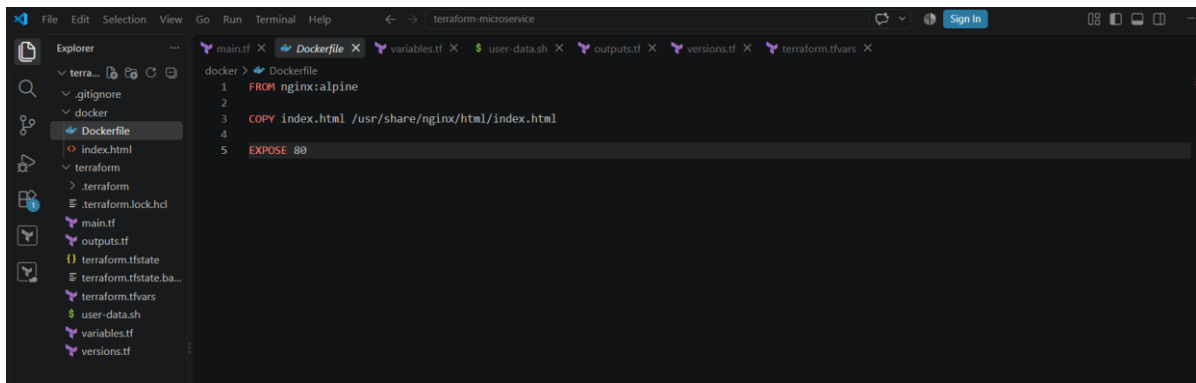
The service is a static page served by nginx. The image is deliberately minimal: it starts from the official `nginx:alpine` base, copies a single HTML file into the default nginx document root, and exposes port 80.

docker/Dockerfile

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

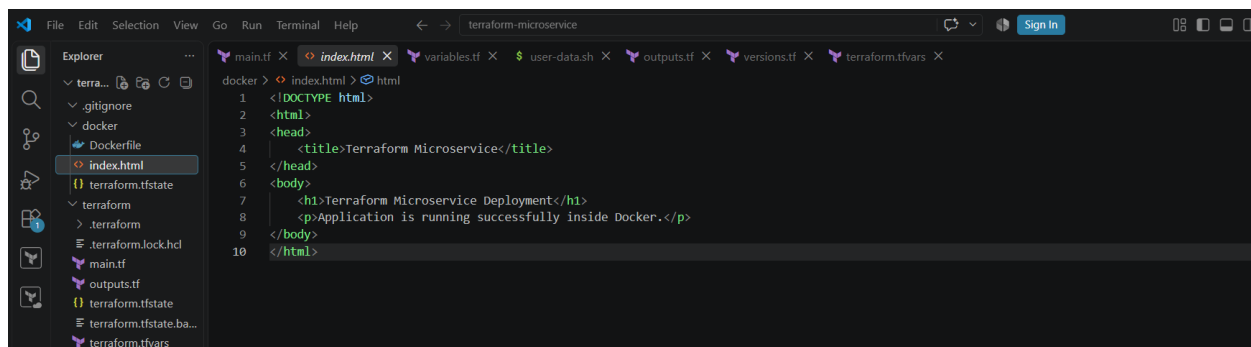
EXPOSE 80
```



3.1 Static Page

docker/index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Terraform Microservice</title>
  </head>
  <body style="background-color:#CFFAFA;">
    <h1>Terraform Microservice Deployment</h1>
    <p>Application is running successfully inside Docker.</p>
  </body>
</html>
```



3.2 Building the Image Locally

As stated in the brief, the image is built locally before Terraform runs, because there is no CI/CD pipeline in this exercise. Terraform then picks up this local image and pushes it to ECR.

```
cd terraform-microservice/docker

docker build -t srihari0310/terraform-microservice:latest .

docker images | findstr terraform-microservice
```

The local tag srihari0310/terraform-microservice:latest is the name referenced by the docker_image resource in main.tf. If this tag does not exist locally, terraform apply will fail at the docker_image step.

4. Deliverable 2 — Terraform Codebase

The complete Terraform configuration is reproduced below, split by file. Within main.tf the resources are grouped with comment banners in the same order in which they are presented here.

4.1 versions.tf / Provider Configuration

Two providers are required: the AWS provider for all cloud resources, and the kreuzwerker/docker provider to tag and push the locally built image into ECR.

main.tf – terraform block and AWS provider

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }

    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

4.2 VPC and Networking

A dedicated VPC is created with DNS support enabled. Two public subnets in different Availability Zones are required by the Application Load Balancer; a single private subnet hosts the EC2 instance.

main.tf – VPC, Internet Gateway and subnets

```
# =====
# VPC
# =====
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "terraform-microservice-vpc"
  }
}

# =====
# INTERNET GATEWAY
# =====
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "terraform-microservice-igw"
  }
}

# =====
# PUBLIC SUBNET 1
# =====
resource "aws_subnet" "public_1" {
```

```

vpc_id          = aws_vpc.main.id
cidr_block      = var.public_subnet_1_cidr
availability_zone = var.availability_zone_1
map_public_ip_on_launch = true

tags = {
  Name = "terraform-public-subnet-1"
}
}

# =====
# PUBLIC SUBNET 2
# =====
resource "aws_subnet" "public_2" {
  vpc_id          = aws_vpc.main.id
  cidr_block      = var.public_subnet_2_cidr
  availability_zone = var.availability_zone_2
  map_public_ip_on_launch = true

  tags = {
    Name = "terraform-public-subnet-2"
  }
}

# =====
# PRIVATE SUBNET
# =====
resource "aws_subnet" "private" {
  vpc_id          = aws_vpc.main.id
  cidr_block      = var.private_subnet_cidr
  availability_zone = var.availability_zone_1
  map_public_ip_on_launch = false

  tags = {
    Name = "terraform-private-subnet"
  }
}
}

```

4.2.1 Public Route Table

```

# =====
# PUBLIC ROUTE TABLE
# =====
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = {
    Name = "terraform-public-route-table"
  }
}

resource "aws_route_table_association" "public_1" {
  subnet_id      = aws_subnet.public_1.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "public_2" {
  subnet_id      = aws_subnet.public_2.id
  route_table_id = aws_route_table.public.id
}

```

4.2.2 NAT Gateway and Private Route Table

The EC2 instance has no public IP, but it still needs outbound internet access in order to install Docker and pull the image from ECR. A NAT Gateway in public subnet 1 provides that egress path while keeping the instance unreachable from the internet.

```
# =====
# NAT GATEWAY
# =====
resource "aws_eip" "nat" {
  domain = "vpc"

  tags = {
    Name = "terraform-nat-eip"
  }
}

resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public_1.id

  depends_on = [
    aws_internet_gateway.main
  ]

  tags = {
    Name = "terraform-nat-gateway"
  }
}

# =====
# PRIVATE ROUTE TABLE
# =====
resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main.id
  }

  tags = {
    Name = "terraform-private-route-table"
  }
}

resource "aws_route_table_association" "private" {
  subnet_id      = aws_subnet.private.id
  route_table_id = aws_route_table.private.id
}
```

4.3 Security Groups

This is where the core security requirement is enforced. The ALB security group accepts HTTP from anywhere. The EC2 security group accepts HTTP only when the source is the ALB security group — not a CIDR range — and accepts SSH only from the single administrator IP supplied through a variable.

main.tf – ALB security group

```
resource "aws_security_group" "alb" {
  name           = "terraform-alb-sg"
  description    = "Security group for Application Load Balancer"
  vpc_id        = aws_vpc.main.id

  ingress {
    description = "HTTP from internet"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "terraform-alb-sg"
  }
}
```

main.tf – EC2 security group

```
resource "aws_security_group" "ec2" {
  name           = "terraform-ec2-sg"
  description    = "Security group for private EC2"
  vpc_id        = aws_vpc.main.id

  # SSH only from your IP
  ingress {
    description = "SSH from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.ssh_allowed_ip]
  }

  # HTTP ONLY from ALB
  ingress {
    description      = "HTTP from ALB"
    from_port        = 80
    to_port          = 80
    protocol         = "tcp"
    security_groups  = [aws_security_group.alb.id]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "terraform-ec2-sg"
  }
}
```

4.4 Docker Registry (Amazon ECR)

```
# =====  
# ECR REPOSITORY  
# =====  
resource "aws_ecr_repository" "microservice" {  
  name           = "terraform-microservice"  
  image_tag_mutability = "mutable"  
  
  image_scanning_configuration {  
    scan_on_push = true  
  }  
  
  tags = {  
    Name = "terraform-microservice"  
  }  
}
```

4.5 IAM Role, Instance Profile and SSH Key Pair

The instance assumes an IAM role that grants read-only access to ECR, so the user-data script can authenticate to the registry without any credentials being stored on the instance. The SSM managed policy is attached as well, which allows Session Manager access to a host that has no public IP.

```
# =====  
# IAM ROLE FOR EC2  
# =====  
resource "aws_iam_role" "ec2" {  
  name = "terraform-microservice-ec2-role"  
  
  assume_role_policy = jsonencode({  
    Version = "2012-10-17"  
    Statement = [  
      {  
        Effect = "Allow"  
        Principal = {  
          Service = "ec2.amazonaws.com"  
        }  
        Action = "sts:AssumeRole"  
      }  
    ]  
  })  
}  
  
# =====  
# ECR READ PERMISSION  
# =====  
resource "aws_iam_role_policy_attachment" "ecr_read" {  
  role      = aws_iam_role.ec2.name  
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"  
}  
  
resource "aws_iam_role_policy_attachment" "ssm" {  
  role      = aws_iam_role.ec2.name  
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"  
}  
  
# =====  
# EC2 INSTANCE PROFILE  
# =====  
resource "aws_iam_instance_profile" "ec2" {  
  name = "terraform-microservice-ec2-profile"  
  role = aws_iam_role.ec2.name  
}  
  
# =====  
# SSH KEY PAIR  
# =====
```

```

resource "aws_key_pair" "microservice" {
  key_name    = "terraform-microservice-key"
  public_key  = file(pathexpand(var.ssh_public_key_path))

  tags = {
    Name = "terraform-microservice-key"
  }
}

```

The key pair is generated locally with `ssh-keygen` and only the public half is uploaded to AWS key storage by Terraform, then attached to the instance — exactly as the brief requires. The private key never leaves the workstation and is excluded by `.gitignore`.

4.6 AMI Lookup and EC2 Instance

The AMI is resolved dynamically rather than hard-coded, so the configuration always launches the latest Amazon Linux 2023 x86_64 image.

```

# =====
# AMAZON LINUX AMI
# =====
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "root-device-type"
    values = ["ebs"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# =====
# EC2 INSTANCE
# =====
resource "aws_instance" "microservice" {
  ami            = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type

  subnet_id              = aws_subnet.private.id
  associate_public_ip_address = false

  vpc_security_group_ids = [
    aws_security_group.ec2.id
  ]

  key_name          = aws_key_pair.microservice.key_name
  iam_instance_profile = aws_iam_instance_profile.ec2.name

  user_data = templatefile("${path.module}/user-data.sh", {
    aws_region      = var.aws_region
    ecr_repository_url = aws_ecr_repository.microservice.repository_url
  })
}

```

```

user_data_replace_on_change = true

tags = {
  Name = "terraform-microservice-ec2"
}

depends_on = [
  aws_nat_gateway.main,
  docker_registry_image.microservice,
  aws_iam_role_policy_attachment.ecr_read,
  aws_iam_role_policy_attachment.ssm
]
}

```

The `depends_on` block is important: the instance must not boot until the NAT Gateway exists (otherwise it has no route out) and until the image has actually been pushed to ECR (otherwise the docker pull in user-data would find an empty repository).

4.7 Application Load Balancer

The ALB is internet-facing and spans the two public subnets. Its target group registers the private EC2 instance on port 80 and health-checks the root path expecting HTTP 200.

```

# =====
# APPLICATION LOAD BALANCER
# =====
resource "aws_lb" "microservice" {
  name          = "terraform-microservice-alb"
  internal      = false
  load_balancer_type = "application"

  security_groups = [
    aws_security_group.alb.id
  ]

  subnets = [
    aws_subnet.public_1.id,
    aws_subnet.public_2.id
  ]

  tags = {
    Name = "terraform-microservice-alb"
  }
}

# =====
# ALB TARGET GROUP
# =====
resource "aws_lb_target_group" "microservice" {
  name     = "terraform-microservice-tg"
  port     = 80
  protocol = "HTTP"

  vpc_id      = aws_vpc.main.id
  target_type = "instance"

  health_check {
    enabled            = true
    healthy_threshold  = 2
    unhealthy_threshold = 3
    timeout            = 5
    interval           = 30
    path               = "/"
    matcher             = "200"
  }
}

```

```

tags = {
  Name = "terraform-microservice-target-group"
}

# =====
# REGISTER EC2 WITH TARGET GROUP
# =====
resource "aws_lb_target_group_attachment" "microservice" {
  target_group_arn = aws_lb_target_group.microservice.arn
  target_id        = aws_instance.microservice.id
  port             = 80
}

# =====
# ALB LISTENER
# =====
resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.microservice.arn

  port        = 80
  protocol    = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.microservice.arn
  }
}

```

4.8 Pushing the Image to ECR with Terraform

This block replaces the CI/CD pipeline. Terraform reads a short-lived ECR authorisation token, references the image that was built locally, re-tags it with the ECR repository URL and pushes it. Because `docker_registry_image` depends on `docker_tag`, which in turn depends on the repository, Terraform sequences registry creation, tagging and push automatically.

```

# =====
# DOCKER PROVIDER
# =====
provider "docker" {
  host = "npipe:///./pipe/docker_engine"
}

# =====
# ECR AUTHENTICATION
# =====
data "aws_ecr_authorization_token" "ecr" {}

# =====
# LOCAL DOCKER IMAGE
# =====
resource "docker_image" "microservice" {
  name          = "srihari0310/terraform-microservice:latest"
  keep_locally = true
}

# =====
# TAG IMAGE FOR ECR
# =====
resource "docker_tag" "microservice" {
  source_image = docker_image.microservice.image_id
  target_image = "${aws_ecr_repository.microservice.repository_url}:latest"

  depends_on = [
    aws_ecr_repository.microservice
  ]
}

```

```
# =====
# PUSH IMAGE TO ECR
# =====
resource "docker_registry_image" "microservice" {
  name = docker_tag.microservice.target_image

  auth_config {
    address = data.aws_ecr_authorization_token.ecr.proxy_endpoint
    username = data.aws_ecr_authorization_token.ecr.user_name
    password = data.aws_ecr_authorization_token.ecr.password
  }

  depends_on = [
    docker_tag.microservice
  ]
}
```

The docker provider host is set to the Windows named pipe `npipes://./pipe/docker_engine` because the deployment was run from a Windows workstation with Docker Desktop. On Linux or macOS this becomes `unix:///var/run/docker.sock`.

4.9 user-data.sh — Pull and Run on the VM

This templated script runs on first boot. It installs Docker and the AWS CLI, authenticates to ECR using the instance's IAM role, pulls the image and runs it on port 80 with a restart policy. Everything is logged to `/var/log/user-data.log` for troubleshooting.

terraform/user-data.sh

```
#!/bin/bash
set -euxo pipefail
exec >>(tee /var/log/user-data.log | logger -t user-data -s 2>/dev/console) 2>&1

echo "==== Starting user-data ====="

echo "Updating packages..."
dnf update -y

echo "Installing Docker and AWS CLI..."
dnf install -y docker awscli

echo "Starting Docker..."
systemctl enable --now docker

echo "Adding ec2-user to docker group..."
usermod -aG docker ec2-user

echo "Logging into ECR..."
aws ecr get-login-password --region ${aws_region} | \
docker login \
  --username AWS \
  --password-stdin ${ecr_repository_url}

echo "Pulling application image..."
docker pull ${ecr_repository_url}:latest

echo "Removing existing container..."
docker rm -f microservice 2>/dev/null || true

echo "Starting microservice..."
docker run -d \
  --name microservice \
  --restart unless-stopped \
  -p 80:80 \
  ${ecr_repository_url}:latest
```

```
echo "Checking Docker..."
docker ps

echo "Testing application..."
curl -f http://localhost/ || true

echo "==== user-data completed ====="
```

4.10 variables.tf

Every environment-dependent value is declared as a variable. Sensible defaults are provided for the network layout; `ssh_public_key_path` and `ssh_allowed_ip` have no default, so Terraform will refuse to run without them being supplied.

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "vpc_cidr" {
  description = "VPC CIDR"
  type        = string
  default     = "10.0.0.0/16"
}

variable "public_subnet_1_cidr" {
  description = "Public subnet 1 CIDR"
  type        = string
  default     = "10.0.1.0/24"
}

variable "public_subnet_2_cidr" {
  description = "Public subnet 2 CIDR"
  type        = string
  default     = "10.0.2.0/24"
}

variable "private_subnet_cidr" {
  description = "Private subnet CIDR"
  type        = string
  default     = "10.0.10.0/24"
}

variable "availability_zone_1" {
  description = "Availability zone 1"
  type        = string
  default     = "us-east-1a"
}

variable "availability_zone_2" {
  description = "Availability zone 2"
  type        = string
  default     = "us-east-1b"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

variable "ssh_public_key_path" {
  description = "SSH public key path"
  type        = string
}
```

```
variable "ssh_allowed_ip" {
  description = "IP allowed to SSH"
  type        = string
}
```

4.11 terraform.tfvars

```
aws_region = "us-east-1"

vpc_cidr          = "10.0.0.0/16"
public_subnet_1_cidr = "10.0.1.0/24"
public_subnet_2_cidr = "10.0.2.0/24"
private_subnet_cidr  = "10.0.10.0/24"

availability_zone_1 = "us-east-1a"
availability_zone_2 = "us-east-1b"

instance_type = "t3.micro"

ssh_public_key_path = "C:/Users/SRIHARI/.ssh/terraform-microservice.pub"
ssh_allowed_ip      = "49.43.224.127/32"
```

4.12 outputs.tf

```
output "vpc_id" {
  description = "VPC ID"
  value       = aws_vpc.main.id
}

output "ec2_private_ip" {
  description = "Private IP address of EC2"
  value       = aws_instance.microservice.private_ip
}

output "ecr_repository_url" {
  description = "ECR repository URL"
  value       = aws_ecr_repository.microservice.repository_url
}

output "load_balancer_dns" {
  description = "Application Load Balancer DNS"
  value       = aws_lb.microservice.dns_name
}

output "application_url" {
  description = "Application URL"
  value       = "http://${aws_lb.microservice.dns_name}"
}
```

5. Deployment Steps and Execution

5.1 Prerequisites

1. An AWS Free Tier account with credentials configured (aws configure).
2. Terraform installed and on the PATH.
3. Docker Desktop running, so the Docker provider can reach the local daemon.
4. An SSH key pair generated locally: `ssh-keygen -t rsa -b 4096 -f ~/.ssh/terraform-microservice`

SSH security requirement

You need your own local SSH key.

If you don't already have one, create it on your local machine:

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/terraform-microservice
```

This creates:

```
terraform-microservice
terraform-microservice.pub
```

Terraform will upload the `.pub` key to AWS.

You will use the private key only for SSH.

5.2 Build the Image Locally

```
cd docker
docker build -t srihari0310/terraform-microservice:latest .
```

5.3 Initialise, Format and Validate

```
cd ../terraform
terraform init
terraform fmt
terraform validate
terraform plan
```

`terraform validate` returns "Success! The configuration is valid.", and `terraform plan` then refreshes state and shows the resources that will be created.

```

PS C:\Users\SRIHARI\OneDrive\Desktop\terraform-microservice\terraform> terraform fmt
PS C:\Users\SRIHARI\OneDrive\Desktop\terraform-microservice\terraform> terraform validate
Success! The configuration is valid.

PS C:\Users\SRIHARI\OneDrive\Desktop\terraform-microservice\terraform> terraform plan
docker_image.microservice: Refreshing state... [id=sha256:5fb661eb91d1e52e56363b92d11b165726bb11ee7dadba2be89d4a5f480f4d97srihari0310/terraform-microservice:latest]
data.aws_ecr_authorization_token.ecr: Reading...
aws_iam_role.ec2: Refreshing state... [id=terraform-microservice-ec2-role]
data.aws_ami.amazon_linux: Reading...
aws_key_pair.microservice: Refreshing state... [id=terraform-microservice-key]
aws_vpc.main: Refreshing state... [id=vpc-0b3eeb028f8a2829c]
aws_eip.nat: Refreshing state... [id=eipalloc-0af94db5ef1aa49]
aws_ecr_repository.microservice: Refreshing state... [id=terraform-microservice]
data.aws_ecr_authorization_token.ecr: Read complete after 1s [id=us-east-1]
docker_tag.microservice: Refreshing state... [id=sha256:5fb661eb91d1e52e56363b92d11b165726bb11ee7dadba2be89d4a5f480f4d97.299582146630.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice:latest]
docker_registry_image.microservice: Refreshing state... [id=sha256:5fb661eb91d1e52e56363b92d11b165726bb11ee7dadba2be89d4a5f480f4d97]
aws_iam_instance_profile.ec2: Refreshing state... [id=terraform-microservice-ec2-profile]
aws_iam_role_policy_attachment.ecr_read: Refreshing state... [id=terraform-microservice-ec2-role/arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly]
data.aws_ami.amazon_linux: Read complete after 3s [id=ami-0d8e124066f7ed4f7]
aws_internet_gateway.main: Refreshing state... [id=igw-074722bab9fed0f43]
aws_subnet.private: Refreshing state... [id=subnet-040959f2d4f9dbefc]
aws_subnet.public_2: Refreshing state... [id=subnet-0c24ccc258f953ee]
aws_subnet.public_1: Refreshing state... [id=subnet-0e2febbf9bc22d0e4]
aws_security_group.alb: Refreshing state... [id=sg-0b784142b02d11c76]
aws_lb_target_group.microservice: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:299582146630:targetgroup/terraform-microservice-tg/a93f2e45573e0fd2]
aws_route_table.public: Refreshing state... [id=rtb-036b54ad1fdfee7fc]
aws_route_table_association.public_1: Refreshing state... [id=rtbassoc-049808bc32ae7a070]

```

5.4 Apply

```
terraform apply -auto-approve
```

```
# to force the instance to be rebuilt and re-run user-data:
terraform apply -replace="aws_instance.microservice"
```

During apply, Terraform refreshes the existing state for every resource — the ECR repository, the pushed image, the VPC and subnets, both security groups, the NAT gateway, the load balancer and the target group — before making changes.

```

PS C:\Users\SRIHARI\OneDrive\Desktop\terraform-microservice\terraform> terraform apply -replace="aws_instance.microservice"
docker_image.microservice: Refreshing state... [id=sha256:5fb661eb91d1e52e56363b92d11b165726bb11ee7dadba2be89d4a5f480f4d97srihari0310/terraform-microservice:latest]
data.aws_ecr_authorization_token.ecr: Reading...
data.aws_ami.amazon_linux: Reading...
aws_key_pair.microservice: Refreshing state... [id=terraform-microservice-key]
aws_vpc.main: Refreshing state... [id=vpc-0b3eeb028f8a2829c]
aws_iam_role.ec2: Refreshing state... [id=terraform-microservice-ec2-role]
aws_eip.nat: Refreshing state... [id=eipalloc-0af94db5ef1aa49]
aws_ecr_repository.microservice: Refreshing state... [id=terraform-microservice]
data.aws_ecr_authorization_token.ecr: Read complete after 1s [id=us-east-1]
docker_tag.microservice: Refreshing state... [id=sha256:5fb661eb91d1e52e56363b92d11b165726bb11ee7dadba2be89d4a5f480f4d97.299582146630.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice:latest]
docker_registry_image.microservice: Refreshing state... [id=sha256:5fb661eb91d1e52e56363b92d11b165726bb11ee7dadba2be89d4a5f480f4d97]
aws_iam_instance_profile.ec2: Refreshing state... [id=terraform-microservice-ec2-profile]
aws_iam_role_policy_attachment.ecr_read: Refreshing state... [id=terraform-microservice-ec2-role/arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly]
data.aws_ami.amazon_linux: Read complete after 2s [id=ami-0d8e124066f7ed4f7]
aws_internet_gateway.main: Refreshing state... [id=igw-074722bab9fed0f43]
aws_subnet.public_2: Refreshing state... [id=subnet-0c24ccc258f953ee]
aws_subnet.private: Refreshing state... [id=subnet-040959f2d4f9dbefc]
aws_subnet.public_1: Refreshing state... [id=subnet-0e2febbf9bc22d0e4]
aws_security_group.alb: Refreshing state... [id=sg-0b784142b02d11c76]
aws_lb_target_group.microservice: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:299582146630:targetgroup/terraform-microservice-tg/a93f2e45573e0fd2]
aws_nat_gateway.main: Refreshing state... [id=nat-00aec497a4b8c81a0]
aws_route_table.public: Refreshing state... [id=rtb-036b54ad1fdfee7fc]
aws_security_group.ec2: Refreshing state... [id=sg-0b073b35ffbc6856]
aws_route_table.private: Refreshing state... [id=rtb-0e3dd0d7c66553186]
aws_lb.microservice: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:299582146630:loadbalancer/app/terraform-microservice-alb/232825f4d01d4644]
aws_route_table_association.public_2: Refreshing state... [id=rtbassoc-08b53d62cd3ab39e]

```

The instance is then replaced, the new instance is created and registered with the target group, and Terraform prints the outputs including the public application URL.

```
terraform > $ user-data.sh
23 docker login \
24 --username $awscli_username \
25 --password stdin $(ecr_repository_url)
26
27 echo "Pulling application image..."
28 docker pull $(ecr_repository_url):latest
29
30 echo "Rebuilding existing container..."

aws_lb_target_group_attachment.microservice: Destruction complete after 1s
aws_instance.microservice: Destroying... [id=i-0a62feb72641f86d]
aws_instance.microservice: Still destroying... [id=i-0a62feb72641f86d, 00m10s elapsed]
aws_instance.microservice: Still destroying... [id=i-0a62feb72641f86d, 00m20s elapsed]
aws_instance.microservice: Still destroying... [id=i-0a62feb72641f86d, 00m30s elapsed]
aws_instance.microservice: Still destroying... [id=i-0a62feb72641f86d, 00m40s elapsed]
aws_instance.microservice: Destruction complete after 42s
docker_registry_image.microservice: Modifying... [id=sha256:5fb661eb91d1e52e56363b92d11b165726bb1ee7dadba2be89da5f480f4d97]
aws_iam_role_policy_attachment.ssm: Creating...
aws_iam_role_policy_attachment.ssm: Creation complete after 1s [id=terraform-microservice-ec2-role/arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore]
aws_instance.microservice: Creating...
aws_instance.microservice: Still creating... [00m0s elapsed]
aws_instance.microservice: Creation complete after 16s [id=i-044c7a6f6bb0c0902]
aws_lb_target_group_attachment.microservice: Creating...
aws_lb_target_group_attachment.microservice: Creation complete after 1s [id=arn:aws:elasticloadbalancing:us-east-1:299582146630:targetgroup/terraform-microservice-tg/a93f2e45e0fd2,1-044c7a6f6bb0c0902,80]

Apply complete! Resources: 3 added, 1 changed, 2 destroyed.

Outputs:

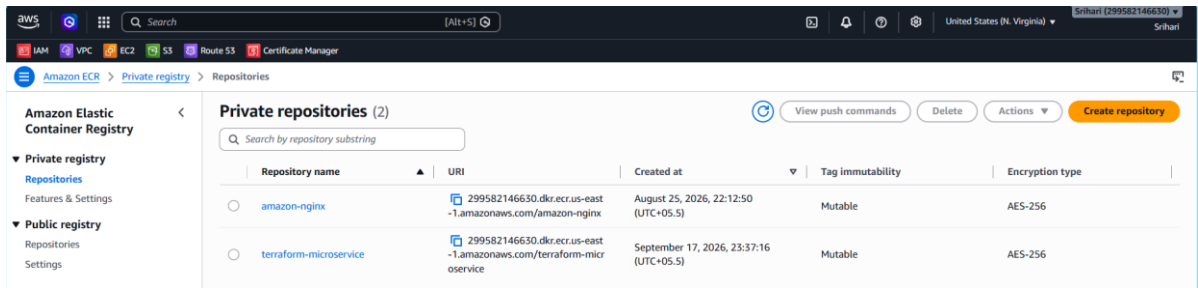
application_url = "http://terraform-microservice-alb-524407649.us-east-1.elb.amazonaws.com"
ec2_private_ip = "10.0.10.167"
ecr_repository_url = "299582146630.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice"
load_balancer_dns = "terraform-microservice-alb-524407649.us-east-1.elb.amazonaws.com"
vpc_id = "vpc-0b3eeb028f8a2829c"
```

6. Verification in the AWS Console

The following screenshots confirm that every requirement in the brief was met by the deployed infrastructure.

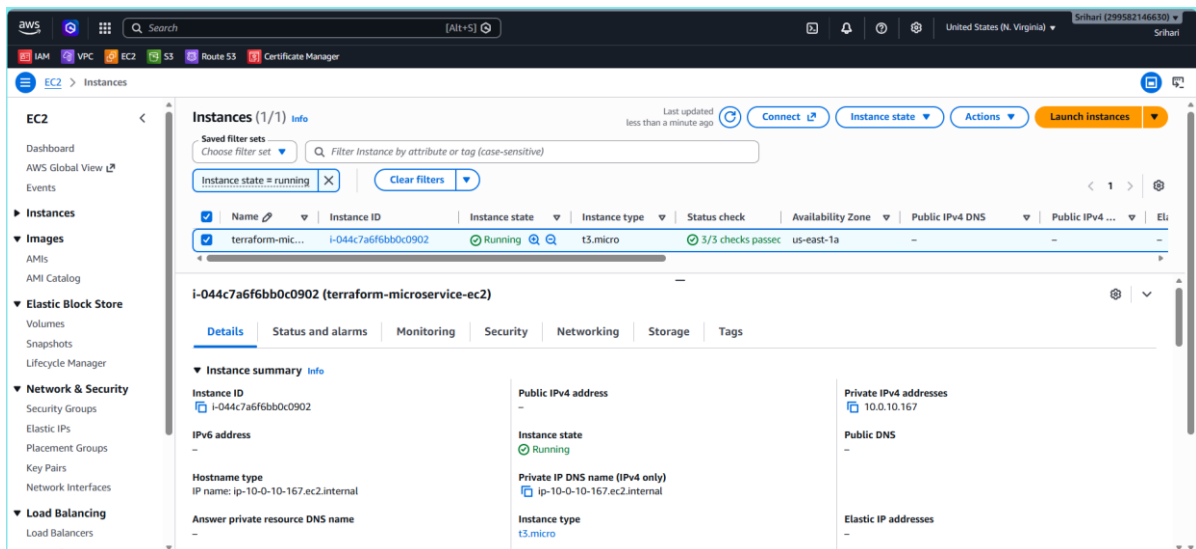
6.1 Docker Registry — Amazon ECR

The repository terraform-microservice was created by Terraform and the locally built image was pushed into it by the docker_registry_image resource.



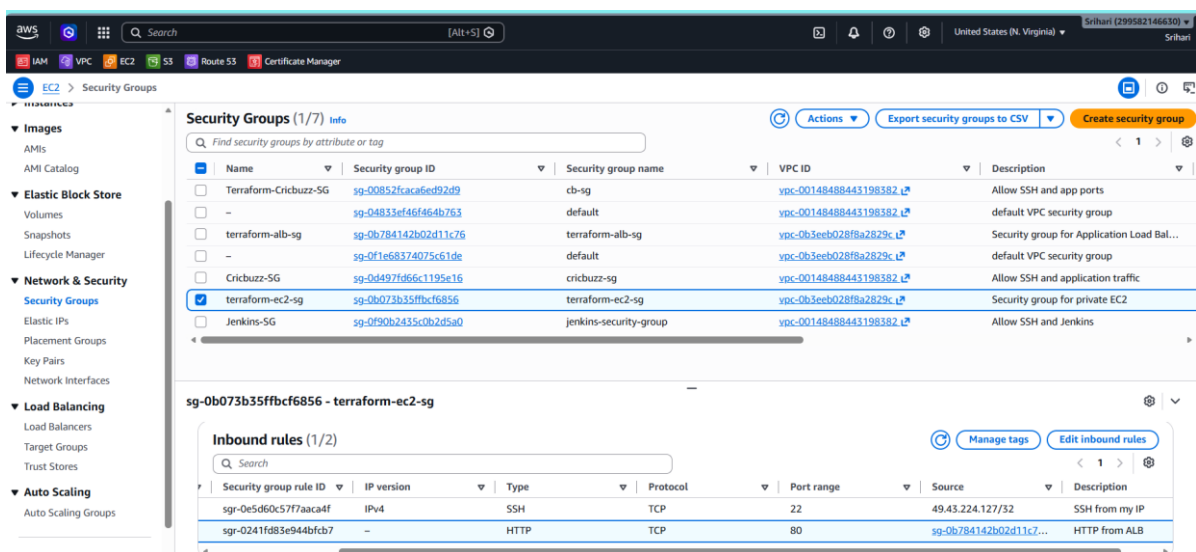
6.2 EC2 Instance — Private, No Public IP

The instance terraform-microservice-ec2 is running as a t3.micro in us-east-1a with 3/3 status checks passing. The Public IPv4 address and Public DNS fields are empty, and the instance only has a private address of 10.0.10.167 — so it cannot be reached directly from the internet.



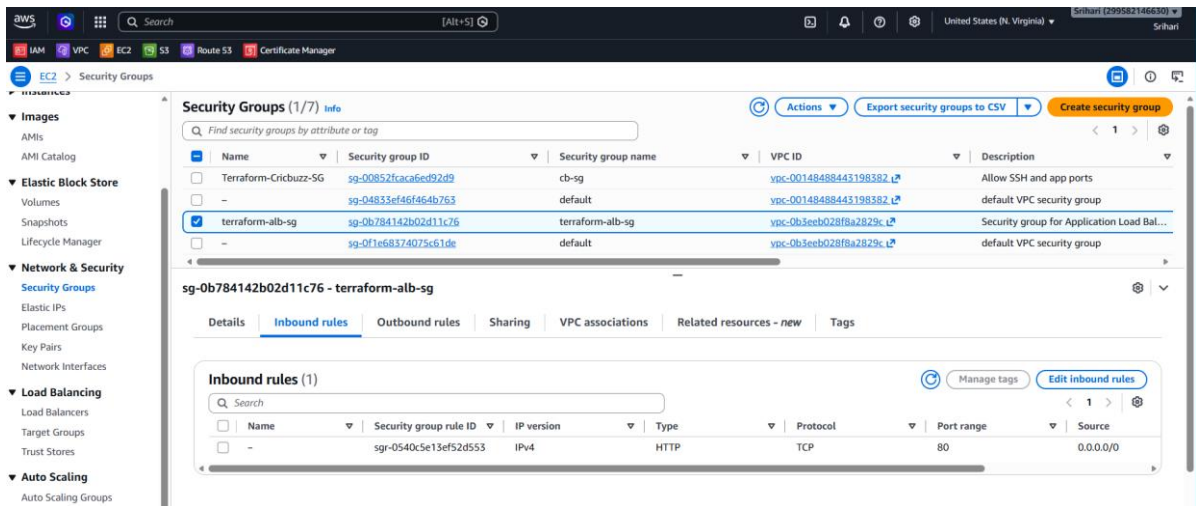
6.3 EC2 Security Group — ALB-only HTTP and Restricted SSH

The inbound rules on terraform-ec2-sg show exactly two entries: SSH on port 22 whose source is the single whitelisted address 49.43.224.127/32, and HTTP on port 80 whose source is the ALB security group ID rather than any CIDR block. This is the enforcement of the "not accessible from the internet except through the Load Balancer" requirement.



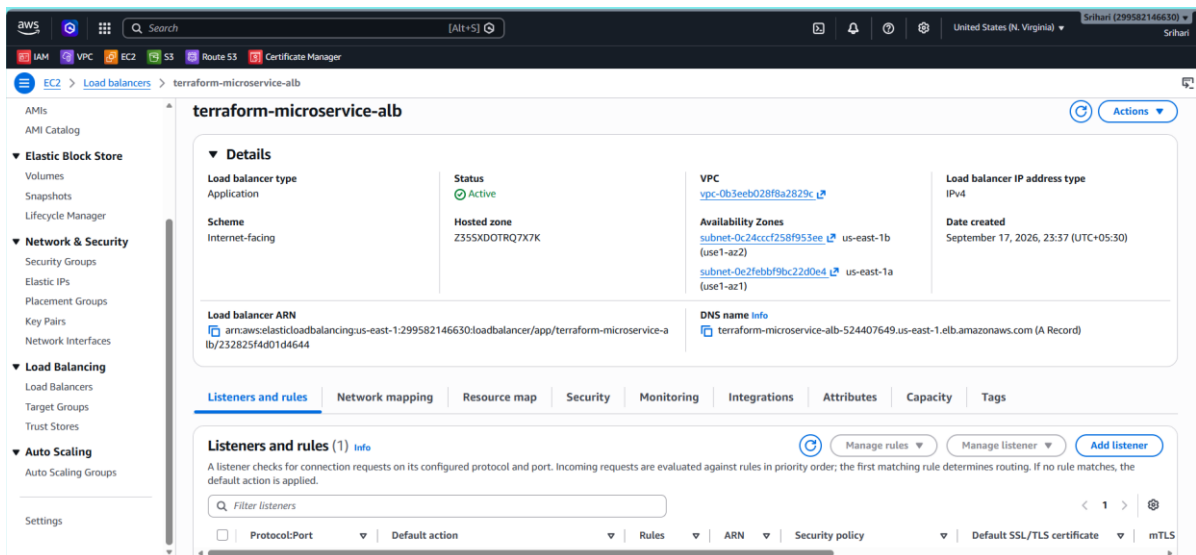
6.4 ALB Security Group

The load balancer's own security group terraform-alb-sg is the only component open to the world, accepting HTTP on port 80 from 0.0.0.0/0.

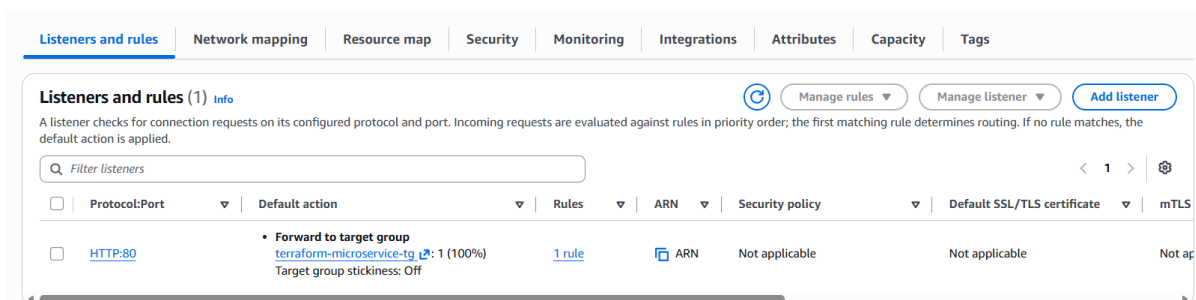


6.5 Application Load Balancer

The ALB terraform-microservice-alb is Active, internet-facing, and attached to both public subnets across us-east-1a and us-east-1b inside the Terraform-created VPC.

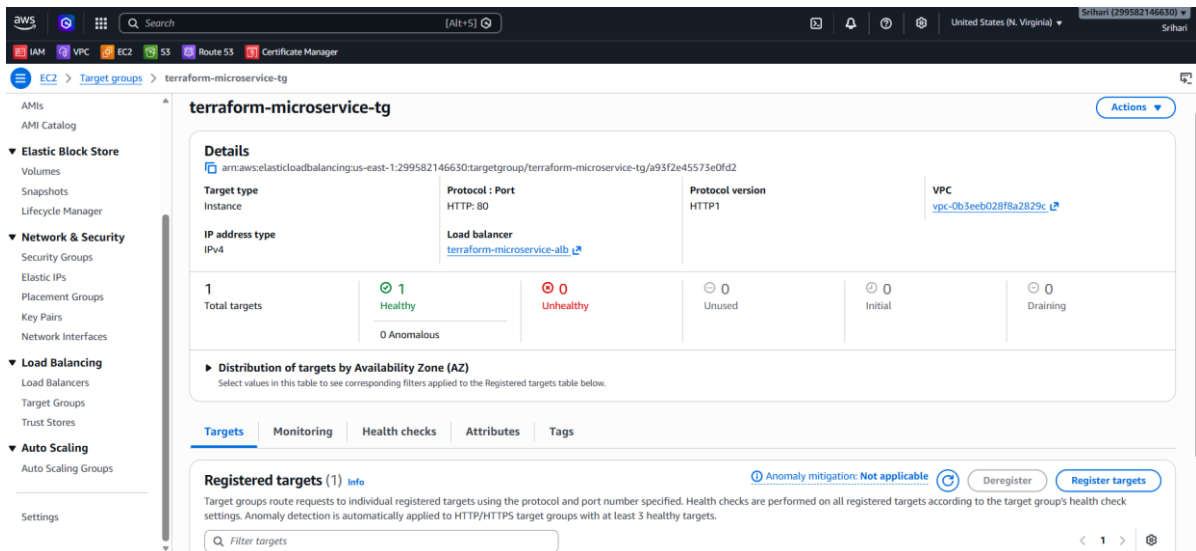


Its single listener accepts HTTP on port 80 and forwards 100% of traffic to the target group.



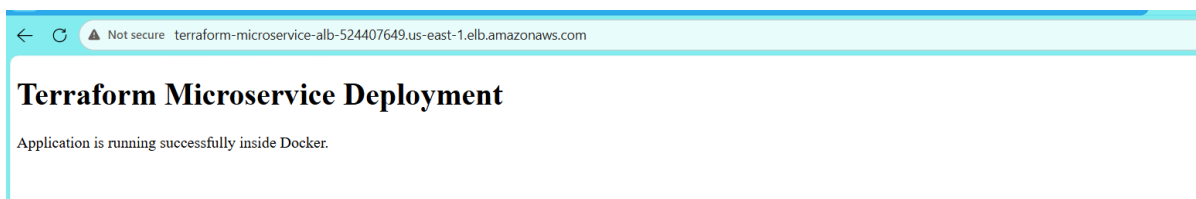
6.6 Target Group Health

The target group reports 1 total target and 1 healthy target with 0 unhealthy, confirming that the container is serving HTTP 200 on the root path and that the ALB can reach the private instance.



6.7 Application Response Through the Load Balancer

Browsing to the load balancer DNS name returns the static page served by nginx from inside the Docker container on the private EC2 instance.



7. Requirements Traceability

The table below maps each requirement from the brief to the implementation and to the evidence in this document.

Requirement	How it is met
Simple web server returning a static page	nginx:alpine serving index.html
Use apache2 or nginx	nginx:alpine base image
Containerized and stored in a Docker registry	Amazon ECR repository terraform-microservice
Deployed on a single VM as a Docker container	One t3.micro EC2 instance; user-data runs docker run
Image pulled from that registry	user-data authenticates to ECR and docker pull
Not internet-accessible; only via Load Balancer	Private subnet, no public IP, HTTP inbound sourced from ALB SG only
SSH only from specific IP addresses	Port 22 inbound restricted to var.ssh_allowed_ip

Requirement	How it is met
Dockerfile deliverable	docker/Dockerfile
Terraform deploys the Docker registry	aws_ecr_repository
Terraform deploys VPC and networking	VPC, 3 subnets, IGW, NAT GW, route tables
Terraform deploys the EC2 instance	aws_instance.microservice
Terraform deploys the Load Balancer	ALB, target group, attachment, listener
Build image locally before Terraform	docker build run manually first
Terraform pushes the local image	docker_image, docker_tag, docker_registry_image
Terraform pulls the image and runs it on the VM	templatefile user-data.sh executed at boot
Locally created SSH key added to cloud key storage and attached	ssh-keygen, aws_key_pair, key_name on the instance
Terraform best practices for structure	main.tf / variables.tf / terraform.tfvars / outputs.tf / versions.tf / user-data.sh
Include tfvars, scripts and helper files	terraform.tfvars, user-data.sh, Dockerfile, index.html, .gitignore
AWS preferred, free tier account	AWS Free Tier, us-east-1, t3.micro

8. Design Decisions and Notes

8.1 Why a NAT Gateway

The instance must have no route in from the internet but still needs a route out to install Docker and reach ECR. A NAT Gateway in the public subnet gives outbound-only connectivity, which is the standard pattern for private application hosts.

8.2 Why an IAM Role Instead of Stored Credentials

Attaching AmazonEC2ContainerRegistryReadOnly through an instance profile means the ECR login inside user-data works without any access keys being written to the instance or baked into the image.

8.3 Why Two Public Subnets

An Application Load Balancer requires subnets in at least two Availability Zones. The application itself still runs on a single instance in one private subnet, as the brief specifies.

8.4 Terraform as a Stand-in for CI/CD

Rather than a pipeline, the Docker provider handles the build artefact lifecycle: the locally built image is tagged for ECR and pushed as part of terraform apply, and depends_on guarantees the push completes before the instance boots and tries to pull.

8.5 Portability Note

The two environment-specific values are the Docker host in the docker provider block and `ssh_public_key_path` in `terraform.tfvars`, both of which reflect a Windows workstation. Changing the host to `unix:///var/run/docker.sock` and the key path to a POSIX path is all that is needed to run this from Linux or macOS.

8.6 Possible Extensions

- Refactor the networking, compute and load balancer blocks into reusable Terraform modules.
- Move state to an S3 backend with DynamoDB locking instead of a local state file.
- Add an HTTPS listener with an ACM certificate and redirect HTTP to HTTPS.
- Replace the single instance with an Auto Scaling Group behind the same target group.